Adaptive LLM Model Routing Using Contextual Bandits with Feedback

Rishi Vel. A
Department of M.Tech Computer Science and Engineering (5 Years Integrated)
Sri Sairam Engineering College
Sai Leo Nagar, West Tambaram-44
Chennai, Tamil Nadu, India
sec22cj027@sairamtap.edu.in

Mrs. Kayalvizhi C
Assistant Professor
Department of M.Tech Computer Science and Engineering (5 Years Integrated)
Sri Sairam Engineering College
Sai Leo Nagar, West Tambaram-44
Chennai, Tamil Nadu, India
[Guide Email ID]

***Abstract*—Applications that can call several large language models (LLMs) must decide which model should answer each query. Larger models tend to produce stronger answers at higher inference latency and computational cost, whereas smaller models are faster and cheaper but may fail on complex tasks. Fixed strategies, such as always using one model or routing by static rules, do not account for differences between individual queries or for changes in model behavior over time. This paper presents a learning-based routing framework that formulates per-query model selection as a contextual-bandit problem and solves it with the Linear Upper Confidence Bound (LinUCB) algorithm. Each query is converted into a compact context vector; LinUCB scores every candidate model by combining an estimated reward with an uncertainty bonus, selects one model, and updates that model's statistics after the outcome is observed. The reward combines normalized response quality, cost, and latency with explicit, documented weights, and feedback can come from task-specific checks, reference answers, a validated judge model, or user ratings. The router is ordinary software and does not modify LLM weights. A controlled initial setup uses three models from one family at different sizes (Qwen3-4B, Qwen3-8B, and Qwen3-14B) served through a common interface. The paper specifies the reward design, dataset strategy, baselines (small-only, large-only, random, and rule-based routing), metrics including cumulative reward and regret, and an experimental protocol covering reward-weight trade-offs, task-category analysis, ablations, workload shift, and sensitivity to the exploration coefficient. Experimental results are not claimed here; the protocol is designed to establish when adaptive routing helps, by how much, and at what cost.**

Keywords*—*Large language models, model routing, contextual bandits, LinUCB, exploration-exploitation, cost-aware inference, feedback-driven learning, LLM serving.

I. Introduction

Large language models are now deployed behind many applications, and an application frequently has access to more than one model. These models differ in response quality, inference latency, and computational or API cost. Sending every request to the largest available model can produce strong answers, but it also increases cost and delay for queries that a smaller model could answer adequately. Sending every request to the smallest model is cheaper and faster, but such a model may struggle with multi-step reasoning, code generation, or other complex tasks.

Prior work has shown that cost and quality can be traded off by choosing among models rather than relying on one. LLM cascades query models in sequence and stop when an answer is judged acceptable [1], while learned routers predict, before generation, which model a query should be sent to [2]-[5]. Benchmarks such as RouterBench formalize the evaluation of such routing systems [6]. Many of these approaches, however, are trained offline on labeled or preference data and remain fixed after deployment, so they do not directly learn from the outcomes they observe in operation.

Contextual bandits provide a natural formulation for learning from operation. At each step, a learner observes a context, chooses one action, and receives a reward only for that action [7]. LinUCB is a widely used contextual-bandit algorithm that models expected reward as a linear function of the context and adds an upper-confidence exploration bonus [8], [9]. In LLM routing, the context describes the incoming query, the actions are candidate models, and the reward expresses the usefulness of the response relative to its cost and latency. Recent studies have begun to apply bandit learning to LLM routing [10], [11], which supports the relevance of this formulation.

This paper addresses the following problem: how can an LLM application select a suitable model for each query while balancing response quality, latency, and cost, and improve its decisions from feedback over time? The proposed system is a LinUCB router that scores each candidate model from query features and its accumulated experience, selects one model, converts the observed outcome into a scalar reward, and updates its statistics. The router is ordinary software and a learning algorithm; it is not itself another LLM, and it does not train or modify the weights of the candidate models. It learns which model to select for which kinds of queries.

The intended contributions are as follows. First, the paper defines an end-to-end routing loop that combines query context, contextual-bandit exploration, observed response quality, latency, and cost. Second, it specifies a reward function that makes the quality-cost-latency trade-off explicit, including consistent treatment of failures, timeouts, and missing feedback. Third, it describes a controlled model pool of three sizes from one model family, a dataset strategy spanning seven task categories, and the measurements to be logged for every decision. Fourth, it defines a reproducible evaluation protocol that compares LinUCB with small-only, large-only, random, and rule-based baselines and analyzes reward-weight sensitivity, task categories, ablations,

workload shift, and exploration settings. These are research goals to be validated experimentally; the paper does not claim that adaptive routing outperforms every baseline.

The study is organized around five research questions. RQ1 asks how query characteristics can be represented as a context vector that is informative for model selection. RQ2 asks whether LinUCB routing achieves higher cumulative reward than fixed, random, and rule-based routing under the same reward definition. RQ3 asks how the choice of reward weights changes the quality-cost-latency operating point and the resulting model-selection distribution. RQ4 asks how quickly routing adapts when the task mix or model behavior changes. RQ5 asks in which task categories adaptive routing helps or struggles, and why.

The remainder of the paper is organized as follows. Section II reviews related work. Section III formulates the problem and describes the model pool and dataset design. Section IV presents the proposed routing framework. Section V describes the experimental setup and evaluation protocol. Section VI defines how results are reported. Section VII summarizes the implementation. Section VIII discusses limitations and risks, and Section IX concludes the paper.

II. Related Work

A. Cost-Aware LLM Selection and Cascades

FrugalGPT [1] studies strategies for reducing the cost of using LLM APIs, including an LLM cascade that sends a query to models in sequence and stops once a scoring function judges a response to be reliable. Hybrid LLM [2] trains a router that decides, for each query, whether a small or a large model should answer, based on predicted query difficulty and a quality level that can be adjusted at test time. These studies establish that a single model is rarely the most efficient choice for every query, but cascades can incur the cost of multiple calls, and two-model routers do not directly address a larger, changing pool of candidates.

B. Learned Routers and Routing Benchmarks

RouteLLM [3] learns a router between a stronger and a weaker model from human preference data with data augmentation. Shnitzer et al. [4] learn model-selection rules from benchmark datasets, and Zooter [5] distills reward-model scores on training queries into a routing function. LLM-Blender [12] instead ensembles the outputs of several models through pairwise ranking and generative fusion, which improves quality but requires running multiple models per query. RouterBench [6] provides an evaluation framework and a large dataset of inference outcomes for comparing routing systems. These approaches are largely supervised: the router is trained offline and then applied, rather than updated from the outcome of each routed request.

C. Contextual Bandits and LinUCB

The multi-armed bandit problem formalizes the exploration-exploitation trade-off, and upper-confidence-bound (UCB) strategies select the action whose optimistic reward estimate is highest [13]. Contextual bandits extend this setting by conditioning the choice on observed features [7]. Li et al. [8] introduced LinUCB for personalized news article recommendation, modeling expected reward as a linear function of context for each action. Chu et al. [9] analyzed regret for contextual bandits with linear payoff functions, and Abbasi-Yadkori et al. [14] derived improved confidence sets for linear stochastic bandits. Because bandit feedback reveals only the reward of the chosen action, offline evaluation requires care; replay-based methods use logged data to evaluate bandit policies without online deployment [15]. For environments whose reward distributions change, discounted and sliding-window UCB variants have been proposed [16].

D. Bandit-Based LLM Routing

MixLLM [10] is a contextual-bandit-based routing system that enhances query embeddings with tags, uses lightweight predictors to estimate response quality and cost for each model, applies a meta-decision maker to balance quality, cost, and latency, and supports continual training. PILOT [11] extends LinUCB for LLM routing by learning a shared embedding space for queries and models from offline human preference data, refining it with online bandit feedback, and adding an online cost policy for budget-constrained routing. These studies show that bandit learning is a viable direction for routing. The present work focuses on a deliberately simple and inspectable design: standard disjoint LinUCB over interpretable query features, an explicit reward that combines quality, cost, and latency, and a controlled same-family model pool, evaluated against simple baselines under a documented protocol.

E. Response Evaluation and Research Gap

Reward quality depends on how responses are scored. Reference-based metrics and unit tests are reliable where they apply, as in benchmark tasks for knowledge, mathematical reasoning, and code generation [17]-[19]. For open-ended responses, strong LLMs can serve as judges, but Zheng et al. [20] also document biases such as position and verbosity effects, which means judge scores must be validated before they are used as rewards. What remains less developed is a reproducible study that connects interpretable query context, contextual-bandit exploration, validated feedback, and an explicit quality-cost-latency reward in a single routing loop, and that reports when such a router improves on simple baselines and when it does not.

III. Problem Formulation and Study Design

A. Contextual-Bandit Formulation

Queries arrive sequentially for t = 1, ..., T. For query q_t, a feature extractor produces a context vector x_t in R^d. The router chooses an action a_t from the candidate set K of models, the chosen model generates a response, and the system observes a scalar reward r_t only for the chosen model. The goal is a policy that maximizes cumulative reward

$$G(T) = \Sigma_{t=1..T} \, r_t(a_t) \qquad (1)$$

or, equivalently, minimizes regret relative to a reference policy. Using an oracle that knows the best model for each query from offline outcomes, regret is

$$Reg(T) = \Sigma_{t=1..T} \, [\, r_t(a^*_t) - r_t(a_t) \,] \qquad (2)$$

This oracle is an analysis reference available only when every model has been run on every query offline; it is not a policy that could be deployed in real time. Here $a^*_t = \text{argmax}_a \, r_t(a)$ is the model with the highest logged reward for query t.

B. Candidate Model Pool

For a controlled initial experiment, the candidate pool consists of three models from the same family at different sizes, drawn from the Qwen3 series [21] (Table I). Using one family reduces confounding differences in tokenizer, prompt format, and training recipe, so that differences between models are driven mainly by scale. The router can later be extended to models from different families or providers. vLLM [22] is the proposed serving layer, and the router communicates with all models through one consistent interface. The exact model versions, quantization, decoding settings, serving configuration, and hardware will be recorded in the experiment report; Table I lists proposed candidates and is not a claim that every configuration has already been tested. Qwen3 models support both a reasoning (thinking) mode and a non-thinking mode [21]; the mode used must be fixed and reported, because it strongly affects output length and latency.

TABLE I
PROPOSED CANDIDATE MODEL POOL

Role	Candidate Model	Nominal Size	Serving	Precision / Quantization	Decoding Settings
Small	Qwen3-4B	4B	vLLM (proposed)	[To be recorded]	[To be recorded]
Medium	Qwen3-8B	8B	vLLM (proposed)	[To be recorded]	[To be recorded]
Large	Qwen3-14B	14B	vLLM (proposed)	[To be recorded]	[To be recorded]

C. Task Categories and Dataset Strategy

The evaluation dataset should represent a range of realistic tasks rather than a single prompt category. Seven categories are used: general question answering, reasoning and multi-step problems, coding and debugging, summarization, text transformation and writing, information extraction, and instruction following. The dataset can combine public benchmark tasks, such as knowledge, mathematical-reasoning, and code-generation benchmarks [17]-[19], with a carefully designed set of application-style queries. The terms and licenses of every dataset must be checked before use. Table II summarizes the quality signal planned for each category.

TABLE II
TASK CATEGORIES AND PLANNED QUALITY SIGNALS

Task Category	Example Source	Primary Quality Signal
General question answering	Knowledge benchmark (e.g., MMLU [17])	Exact match against reference answer
Reasoning / multi-step problems	Math word problems (e.g., GSM8K [18])	Final-answer match against reference
Coding and debugging	Code benchmark (e.g., HumanEval [19])	Unit-test pass/fail
Summarization	Application-style documents	Rubric score by validated judge
Text transformation / writing	Application-style prompts	Rubric score by validated judge
Information extraction	Application-style documents with labels	Field-level match against reference
Instruction following	Application-style prompts with constraints	Constraint checks and rubric score

D. Logged Fields and Measurements

Every query-model interaction is logged with the fields in Table III. Prompts, model versions, decoding settings, evaluation rules, and hardware configuration are kept consistent across comparisons.

TABLE III
LOGGED FIELDS FOR EACH QUERY-MODEL INTERACTION

Field	Purpose
query_id	Unique identifier for the example
prompt	User query sent to each model
task_category	Known task label, if available
context_features	Feature vector used by the router
model_id	Model that generated the response
response	Model output
quality_score	Reference-based, rubric-based, or judge-based score
latency_ms	Measured response latency
input_tokens / output_tokens	Token counts, when available
cost	Calculated cost or defined compute-cost proxy
feedback	User or evaluator feedback, if available
reward	Reward calculated from the chosen metrics

E. Building the Evaluation Dataset

The initial evaluation dataset is built in six steps: (1) select a representative set of queries and assign task categories; (2) run each query through each candidate model, where feasible, to obtain comparable measurements; (3) evaluate all outputs with the same task-specific criteria; (4) record latency, token counts, cost, failures, and quality; (5) manually inspect a sample of automated scores to validate them; and (6) separate development and tuning data from held-out test data to reduce leakage. Running every model on every query produces a full-information log that supports offline comparison and oracle-based regret. In online bandit learning, by contrast, the router observes only the outcome of the model it selected, unless additional model outputs are deliberately collected.

IV. Proposed Routing Framework

The proposed framework places a contextual-bandit router between the application and the candidate models. Fig. 1 shows the loop: the query is converted into features, LinUCB scores each candidate, one model generates the response, the response is evaluated, a reward is computed, and the chosen model's statistics are updated before the next query.

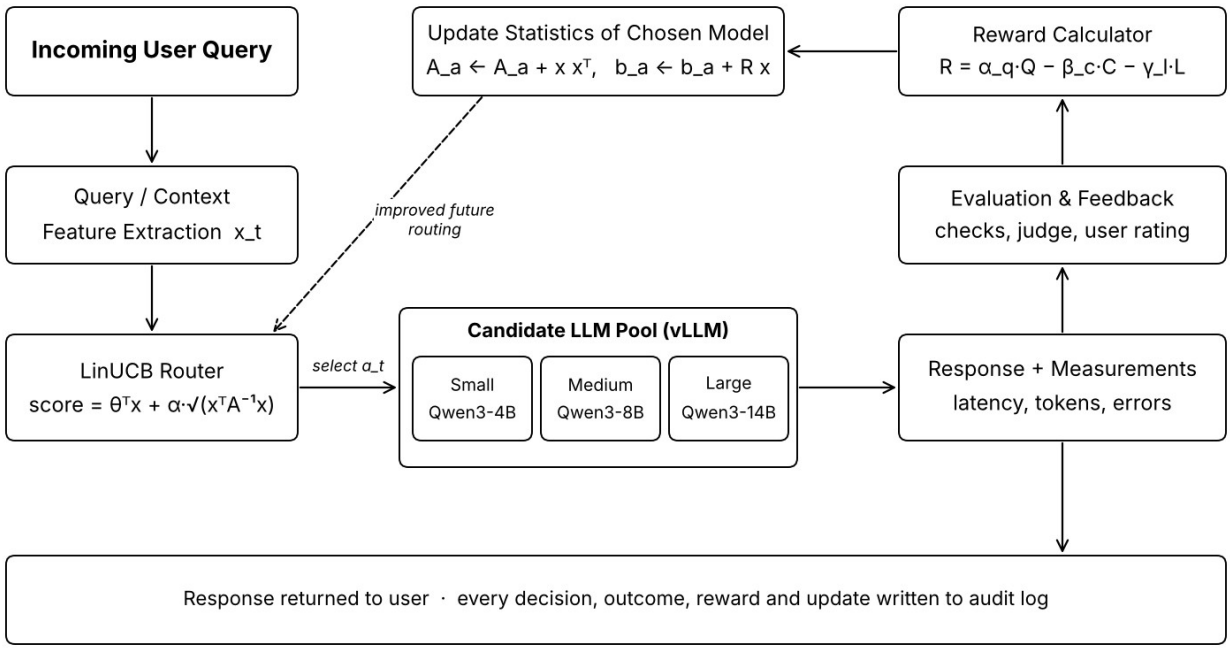


Fig. 1. Proposed adaptive LLM routing loop with contextual-bandit feedback.

A. Query Feature Extraction

The feature extractor converts each query into a compact numerical context x_t. Candidate features include normalized prompt length in tokens, a one-hot task-category indicator, indicators for code-related content, an estimated-difficulty signal, and optionally a low-dimensional projection of a sentence embedding [23]. A constant bias term is appended. Features are computed only from information available before generation, and the feature set is fixed on the development split before the main comparison. The contribution of each feature group is measured by ablation (Section V).

B. LinUCB Scoring

The router uses disjoint LinUCB [8], which keeps separate statistics for each model a: a d×d matrix A_a and a d-dimensional vector b_a, initialized as

$$A_a = \lambda \cdot I_d, \quad b_a = 0_d \tag{3}$$

where $\lambda > 0$ is a regularization parameter ($\lambda = 1$ corresponds to the original formulation [8]). The estimated relationship between features and reward for model a is

$$\theta_a = A_a^{-1} b_a \tag{4}$$

and the score of model a for context x is

$$p_a = \theta_a^{\top} x + \alpha \cdot \sqrt{x^{\top} A_a^{-1} x} \tag{5}$$

The first term estimates the expected reward of model a for this query; the second term is an exploration bonus that is large when the router has little experience with similar contexts for that model. The coefficient α controls exploration. The router selects $a_t = \arg\max_a p_a$, breaking ties by a fixed, documented rule.

C. Reward Function

The reward translates the competing objectives into one value. The starting formulation is

$$R = \alpha_q \cdot Q - \beta_c \cdot \tilde{C} - \gamma_l \cdot \tilde{L} \tag{6}$$

where Q in [0, 1] is the normalized response-quality score, $\tilde{C}$ is normalized cost, $\tilde{L}$ is normalized latency, and α_q, β_c, and γ_l are non-negative weights expressing the project's priorities. The quality weight α_q is distinct from the LinUCB exploration coefficient α. Cost is either monetary cost or a clearly defined compute-cost proxy, such as GPU-seconds per request. Cost and latency are normalized with fixed ranges determined on the development split and clipped to [0, 1]:

$$\tilde{C} = \mathrm{clip}\left((C - C_{min}) / (C_{max} - C_{min}), 0, 1 \right) \tag{7}$$

with latency normalized in the same way. The weights and normalization ranges are fixed and documented before the main comparison. Invalid responses, timeouts, and failed requests receive Q = 0 together with their measured cost and latency, so a cheap but unusable response cannot receive a high reward. If an additional utility or user-feedback term is included, it is defined explicitly so that the same signal is not counted twice. The handling of missing feedback is also defined explicitly: an observation without a validated quality signal is either deferred until feedback arrives or excluded from the update, and the chosen rule is reported.

D. Feedback and Learning Update

Feedback can come from explicit user ratings or corrections, unit tests for coding tasks, exact-match or reference-based metrics, rubric-based evaluation by a judge model [20], and latency, cost, timeout, and error

measurements. After the reward r_t for the chosen model a_t is computed, only that model's statistics are updated:

$$A_a \leftarrow A_a + x_t x_t^T, \quad b_a \leftarrow b_a + r_t x_t \quad (a = a_t) \quad (8)$$

Over many requests, the router refines its estimate of which model works best for which contexts. Because a noisy judge or an inconsistent rubric can teach the router the wrong behavior, automated scores are validated against a manually inspected sample before they are used as rewards.

E. Exploration and Exploitation

Exploitation chooses the model currently expected to give the best reward for a kind of query; exploration tries a model whose outcome is uncertain, because it may perform better than current estimates suggest. Excessive exploration wastes resources and temporarily lowers quality or raises cost, whereas too little exploration can prevent the router from discovering better choices. The exploration coefficient α is therefore treated as a key hyperparameter, tuned on development data and analyzed for sensitivity.

F. Operational Safeguards

A production implementation needs explicit handling of timeouts, unavailable models, missing or delayed feedback, retries, and malformed outputs. A fallback policy can be added, for example retrying with a larger model after a failure, but any such policy is evaluated and documented rather than silently changing the experiment. Every decision is written to an audit log containing the context, selected model, outcome, reward, and update.

Algorithm 1. LinUCB-Based Adaptive LLM Routing

1) For every candidate model a: initialize $A_a = \lambda I_d$ and $b_a = 0_d$.
2) For each incoming query q_t:
   3) Extract the context vector x_t = extract_features(q_t).
   4) For each model a: compute $\theta_a = A_a^{-1} b_a$ and $p_a = \theta_a^T x_t + \alpha\sqrt{(x_t^T A_a^{-1} x_t)}$.
   5) Select a_t = argmax_a p_a.
   6) Call model a_t; record response, latency, token usage, cost, and errors.
   7) Evaluate the response to obtain quality Q (checks, reference, validated judge, or user feedback).
   8) Compute reward $r_t = \alpha_q \cdot Q - \beta_c \cdot \tilde{C} - \gamma_l \cdot \tilde{L}$ (failures receive Q = 0).
   9) For $a = a_t$ only: update $A_a \leftarrow A_a + x_t x_t^T$ and $b_a \leftarrow b_a + r_t x_t$.
   10) Log context, model, outcome, reward, and update; return the response.

V. Experimental Setup and Evaluation

A. Baseline Routing Policies

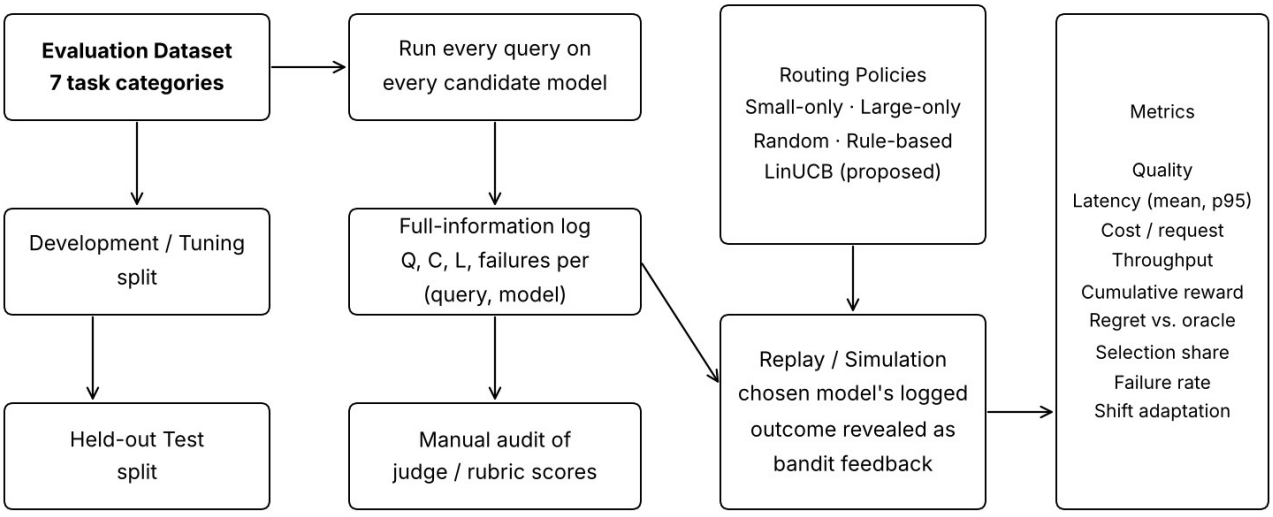
LinUCB is compared with four simple strategies that use the same model pool, prompts, and reward definition (Table IV). The rule-based baseline routes by task category or estimated difficulty using rules fixed on the development split.

TABLE IV
ROUTING POLICIES COMPARED

Policy	Decision Rule	Learns from Feedback
Small-only	Always select Qwen3-4B	No
Large-only	Always select Qwen3-14B	No
Random	Select a model uniformly at random (fixed seed)	No
Rule-based	Route by task category or estimated difficulty using fixed rules	No
LinUCB (proposed)	Select argmax of Eq. (5); update with Eq. (8)	Yes

B. Evaluation Protocol

The evaluation follows the design in Fig. 2. Because every query in the evaluation set is run on every candidate model, each policy can be evaluated on the same query stream by revealing only the logged outcome of the model it selects. This full-information replay gives every policy identical inputs and also makes the oracle in Eq. (2) computable [15]. LinUCB hyperparameters (α, λ) and the reward weights are tuned only on the development split; final comparisons are reported on the held-out split. An online run, in which the router observes only the outcome of the model it actually calls, is used to confirm that conclusions hold under true bandit feedback and live latency.



α, λ and reward weights tuned on the development split; final comparison reported on the held-out split only

Fig. 2. Evaluation design: full-information logging, replay-based policy comparison, and held-out reporting.

C. Evaluation Metrics

Table V lists the metrics. Generation settings, hardware, serving configuration, concurrency, token limits, temperature, and evaluation method are reported alongside every result. Throughput is measured under a stated load, and latency is reported as both mean and tail (p95) values.

TABLE V
EVALUATION METRICS

Metric	What It Measures
Response quality	Correctness or task-specific usefulness
Average and p95 latency	Speed, including tail behavior
Cost per request	Monetary or compute cost under a stated definition
Throughput	Requests handled per unit time under a stated load
Cumulative reward	Total reward accumulated by the routing policy, Eq. (1)
Regret	Difference from the offline oracle reference, Eq. (2)
Model-selection distribution	How frequently each candidate model is selected
Failure rate	Timeouts, errors, and unusable outputs
Adaptation after a shift	How quickly performance responds to changes in task mix or model behavior

D. Planned Experiments

Seven experiments address the research questions (Table VI). Each varies one factor while holding the others fixed.

TABLE VI
PLANNED EXPERIMENTS

Experiment	Factor Varied	Question Addressed
E1. Overall comparison	Routing policy	Does LinUCB improve on small-only, large-only, random, and rule-based routing? (RQ2)
E2. Quality-cost trade-off	Reward weights α_q, β_c, γ_l	How do priorities shift the operating point and model mix? (RQ3)
E3. Task-category analysis	Task category	Where does adaptive routing help or struggle? (RQ5)
E4. Ablation	Feature groups; feedback components	Which features and feedback signals matter? (RQ1)
E5. Workload shift	Mix of task categories over time	How quickly does routing adapt? (RQ4)
E6. Sensitivity	Exploration α; regularization λ	How robust are results to key hyperparameters?
E7. Reliability	Random seed and query order	How variable are the results across runs?

E. Statistical Analysis

Each policy is run over several random seeds and query orderings, because the results of online learning depend on the order in which queries arrive. Results are reported with uncertainty intervals, for example bootstrap intervals over queries [24], rather than single point estimates. Differences between LinUCB and a baseline are interpreted only when they are consistent across runs.

F. Evaluation Integrity

Held-out test queries are not used to select features, reward weights, normalization ranges, or hyperparameters. Automated quality scores are spot-checked against manual judgments, and the agreement rate is reported. The analysis is designed to establish when adaptive routing helps, by how much, and what trade-offs it introduces; it does not assume that adaptive routing will outperform every baseline.

VI. Results and Analysis

The experiments in Section V have not yet been completed. This section specifies how results will be reported. Tables VII and VIII define the reporting format; cells marked [TBD] are placeholders to be replaced with measured values, and no numerical result is claimed in this version of the paper.

A. Overall Comparison

TABLE VII
OVERALL POLICY COMPARISON ON THE HELD-OUT SPLIT (FORMAT)

Policy	Mean Quality	Mean Latency (ms)	p95 Latency (ms)	Cost / Request	Failure Rate	Cumulative Reward	Regret
Small-only	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Large-only	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Random	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
Rule-based	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
LinUCB (proposed)	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

Table VII will report each policy's mean and uncertainty interval across runs. The comparison of interest is whether LinUCB reaches a better quality-cost-latency balance than fixed routing, not whether it maximizes quality alone; large-only routing is expected to be a strong quality reference, and small-only routing a strong cost and latency reference.

B. Model-Selection Behavior by Task Category

TABLE VIII
LINUCB MODEL-SELECTION SHARE BY TASK CATEGORY (FORMAT)

Task Category	Qwen3-4B	Qwen3-8B	Qwen3-14B	Mean Reward vs. Best Fixed Policy
General question answering	[TBD]	[TBD]	[TBD]	[TBD]
Reasoning / multi-step	[TBD]	[TBD]	[TBD]	[TBD]
Coding and debugging	[TBD]	[TBD]	[TBD]	[TBD]
Summarization	[TBD]	[TBD]	[TBD]	[TBD]
Text transformation / writing	[TBD]	[TBD]	[TBD]	[TBD]
Information extraction	[TBD]	[TBD]	[TBD]	[TBD]
Instruction following	[TBD]	[TBD]	[TBD]	[TBD]

Table VIII will show whether the router learns category-dependent behavior, for example whether it reserves the larger model for categories where the smaller model frequently fails. Categories in which LinUCB does not improve on the best fixed policy will be reported and analyzed rather than omitted.

C. Trade-off, Shift, and Sensitivity Analyses

For experiment E2, results will be presented as quality-versus-cost and quality-versus-latency curves across reward-weight settings. For E5, cumulative reward and selection share will be plotted over time around the point at which the task mix changes. For E4 and E6, the change in cumulative reward relative to the full configuration will be reported for each removed feature group, feedback component, and hyperparameter setting.

VII. System Implementation

A. Router Service and Model Interface

The router is implemented as a service between the application and the serving layer (Fig. 3). It contains the feature extractor, the LinUCB policy state (A_a and b_a for each model), a model interface, and a reward calculator with a feedback intake. The model interface sends the request to the selected model through vLLM [22], which provides one consistent endpoint per model, and records latency, token usage, and errors. Implementation proceeds in six phases: establish the model interface, build the evaluation pipeline, implement the baselines, implement LinUCB, add feedback, and evaluate.

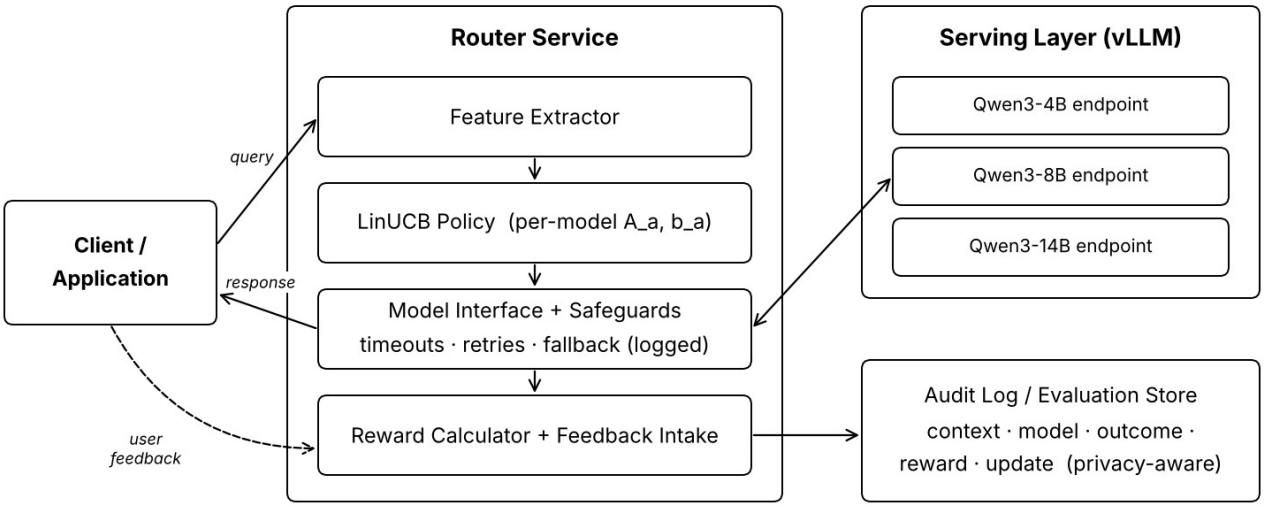


Fig. 3. Implementation view: router service, vLLM serving layer, and audit log.

B. Logging, Reproducibility, and Privacy

Every routing decision is written to an audit log with the context, selected model, outcome, reward, and update, so that any decision can be reconstructed and the experiment can be replayed. Model versions, decoding settings, hardware, and the reward configuration are stored with each run. Sensitive user content is not stored unless it is necessary and authorized; where possible, the log keeps derived features and scores instead of raw prompts.

VIII. Discussion

A. Learning Which Model to Use, Not One Best Model

The central idea of the framework is not to find one universally best LLM. It is to learn which available model is the most suitable choice for a particular query under explicit quality, cost, and latency objectives. Because the router learns only model-selection statistics, it is lightweight, inspectable, and independent of the candidate models' weights.

B. Reward Design and Feedback Quality

The reward determines what the router learns. Different reasonable weights can change which model is preferred, so conclusions are reported across several weight settings

rather than one. Feedback quality matters as much as the algorithm: automated judges may be biased or inconsistent [20], and a noisy reward can teach the router the wrong behavior. Validating judge scores against manual inspection is therefore part of the method, not an optional step.

C. Limitations and Risks

The framework has several known limitations. At cold start the router has little information, and exploration can temporarily reduce quality or increase cost. Weak query features may not predict which model will perform well. Provider updates, quantization, or serving changes can invalidate past observations; standard LinUCB assumes stationary rewards, so sliding-window or discounted variants [16] may be needed under persistent drift. Network and system load can obscure model-specific latency. Results on one dataset or model family may not transfer to others. Finally, logging prompts and feedback requires appropriate data handling.

IX. Conclusion and Future Work

This paper presented a framework for adaptive LLM model routing that formulates per-query model selection as a contextual-bandit problem and uses LinUCB to balance response quality, latency, and cost. The router extracts query features, scores each candidate model with an estimated reward and an uncertainty bonus, selects one model, converts the observed outcome into a reward, and updates its statistics, so that routing can improve from feedback over time without modifying the candidate models.

The paper also specified a controlled same-family model pool, a seven-category dataset strategy, an explicit reward function, simple baselines, and an evaluation protocol based on full-information logging, replay, held-out reporting, and repeated runs. The pipeline can be summarized as: query, feature extraction, LinUCB scoring, model selection, response generation, quality, cost, and latency measurement, reward calculation, feedback update, and improved future routing. Whether and where this improves on fixed routing is a question for the planned experiments, whose results will be reported in the format defined in Section VI.

Future work includes completing the planned experiments; extending the pool to models from different families and providers; evaluating non-stationary bandit variants for changing workloads [16]; adding budget constraints to the selection rule, as explored in recent bandit-based routing work [11]; handling delayed and partial feedback; and comparing LinUCB with other contextual-bandit algorithms such as Thompson sampling or neural bandits.

REFERENCES

[1] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[2] D. Ding et al., "Hybrid LLM: Cost-efficient and quality-aware query routing," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[3] I. Ong et al., "RouteLLM: Learning to route LLMs from preference data," in Proc. Int. Conf. Learning Representations (ICLR), 2025.

[4] T. Shnitzer et al., "Large language model routing with benchmark datasets," arXiv:2309.15789, 2023.

[5] K. Lu et al., "Routing to the expert: Efficient reward-guided ensemble of large language models," in Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL), 2024.

[6] Q. J. Hu et al., "RouterBench: A benchmark for multi-LLM routing system," arXiv:2403.12031, 2024.

[7] T. Lattimore and C. Szepesvári, Bandit Algorithms. Cambridge, U.K.: Cambridge Univ. Press, 2020.

[8] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. Conf. World Wide Web (WWW), 2010, pp. 661-670, doi: 10.1145/1772690.1772758.

[9] W. Chu, L. Li, L. Reyzin, and R. E. Schapire, "Contextual bandits with linear payoff functions," in Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS), PMLR vol. 15, 2011, pp. 208-214.

[10] X. Wang et al., "MixLLM: Dynamic routing in mixed large language models," in Proc. 2025 Conf. Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL), vol. 1, 2025, pp. 10912-10922, doi: 10.18653/v1/2025.naacl-long.545.

[11] P. Panda, R. Magazine, C. Devaguptapu, S. Takemori, and V. Sharma, "Adaptive LLM routing under budget constraints," in Findings of the Association for Computational Linguistics: EMNLP 2025, 2025, pp. 23934-23949.

[12] D. Jiang, X. Ren, and B. Y. Lin, "LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion," in Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL), 2023.

[13] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," Machine Learning, vol. 47, no. 2-3, pp. 235-256, 2002.

[14] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," in Advances in Neural Information Processing Systems (NeurIPS), vol. 24, 2011.

[15] L. Li, W. Chu, J. Langford, and X. Wang, "Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms," in Proc. 4th ACM Int. Conf. Web Search and Data Mining (WSDM), 2011.

[16] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in Proc. 22nd Int. Conf. Algorithmic Learning Theory (ALT), LNCS vol. 6925, 2011, pp. 174-188.

[17] D. Hendrycks et al., "Measuring massive multitask language understanding," in Proc. Int. Conf. Learning Representations (ICLR), 2021.

[18] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.

[19] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[20] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, vol. 36, 2023.

[21] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.

[22] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. 29th ACM Symp. Operating Systems Principles (SOSP), 2023, doi: 10.1145/3600006.3613165.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982-3992.

[24] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman & Hall, 1993.